Big Data Coursework 1: Search Engine Design

Dom Richards

In this coursework you'll be completing a small search engine that retrieves information about two of the great Williams: Shatner and Shakespeare.

1) Initial Setup

We'll start by loading some packages:

```
required_pkgs <- c(
  "sparklyr", "dplyr", "purrr", "rvest", "stringr", "tibble", "xml2", "rlang"
)
missing_pkgs <- setdiff(required_pkgs, rownames(installed.packages()))

if (length(missing_pkgs) > 0) {
  install.packages(
    missing_pkgs,
    repos = "https://cloud.r-project.org",
    dependencies = TRUE
  )
}

library(sparklyr)
library(dplyr)
library(purrr)
library(rvest)
library(stringr)
library(tibble)
library(rlang)

# Disable Arrow to avoid serialization warnings
options(sparklyr.arrow = FALSE)
```

Once the packages are loaded, we can establish a connection to a local Spark instance:

```
# Choose a safe, writable base directory
base_dir <- getwd()

# Fall back to tempdir() if current directory is not writable
if (!file.access(base_dir, 2) == 0) {
  base_dir <- tempdir()
}

# Construct platform-agnostic log path
log_dir <- file.path(base_dir, "logs")
```

```r
log_file <- file.path(log_dir, "log4j.spark.log")

# Create directories and file safely
if (!dir.exists(log_dir)) dir.create(log_dir, recursive = TRUE, showWarnings = FALSE)
if (!file.exists(log_file)) file.create(log_file)

# Configure Spark
cfg <- spark_config()
cfg$sparklyr.arrow <- FALSE
cfg$sparklyr.connect.enableHive <- FALSE

# Optionally expose the log directory for Spark
Sys.setenv(SPARK_LOG_DIR = log_dir)

sc <- spark_connect(master = "local[*]", config = cfg, version = "4.0.1")
```

2) Load Wikipedia pages

Now we need to pull the Wikipedia pages for William Shatner and William Shakespeare. We'll use a cache directory to store the downloaded HTML files, so that we don't have to re-download them every time we run the code:

```r
# Use OS-agnostic temp directory
cache_dir <- file.path(tempdir(), "woogle")
```

With the cache location set, we can download and parse the pages:

```r
urls <- c(
  shatner     = "https://en.wikipedia.org/wiki/William_Shatner",
  shakespeare = "https://en.wikipedia.org/wiki/William_Shakespeare"
)

# Ensure the cache directory exists
if (!dir.exists(cache_dir)) {
  dir.create(cache_dir, recursive = TRUE)
}

# Pull and extract text from the HTML <p> (i.e., paragraph) tags, caching pages locally
texts_info <- purrr::imap(urls, ~ {
  cache_path <- file.path(cache_dir, paste0(.y, ".html"))
  status <- "loaded from cache"

  if (!file.exists(cache_path)) {
    download.file(.x, cache_path, mode = "wb", quiet = TRUE)
    status <- "downloaded"
  }

  cache_path_abs <- normalizePath(cache_path, winslash = "/", mustWork = FALSE)

  text <- read_html(cache_path) %>%
    html_elements("p") %>%
    html_text2() %>%
```

```
    paste(collapse = " ")

  list(
    name = .y,
    text = text,
    cache_path = cache_path_abs,
    status = status
  )
})

texts_df <- purrr::map_dfr(texts_info, ~ tibble(name = .x$name, text = .x$text))

status_tbl <- purrr::map_dfr(texts_info, ~ tibble(
  name = .x$name,
  cache_path = .x$cache_path,
  status = .x$status
))

texts_df
status_tbl
```

Your output should look something like this:

```
texts_df

## # A tibble: 2 x 2
## name text
## <chr> <chr>
## 1 shatner " William Shatner[1][2]OC (born March 22, 1931) is a Canadian act~
## 2 shakespeare " William Shakespeare[a] (c. 23 April 1564[b] - 23 April 1616)[c]~

status_tbl

## # A tibble: 2 x 3
## name cache_path status
## <chr> <chr> <chr>
## 1 shatner /tmp/woogle/shatner.html loaded from cache
## 2 shakespeare /tmp/woogle/shakespeare.html loaded from cache
```

`texts_df` gives us a compact table with each author's name paired with the scraped text. Also, `status_tbl` confirms whether the content was downloaded during this run or loaded from our cache, along with the absolute file path so you can inspect the HTML if needed.

Note that the purrr::imap() function maps an anonymous function over the values and names of the urls vector. For example, in `texts_info`, the syntax ~ { starts an anonymous function where .x refers to each URL value (e.g., "https://en.wikipedia.org/wiki/William_Shatner") and .y refers to its name (e.g., "shatner"). This allows us to use the name for creating cache filenames while downloading from the corresponding URL.

3) Copy to Spark

With the documents staged in R, we next mirror them into Spark so we can use Spark ML transformers. `sdf_copy_to()` creates a new Spark table while keeping a reference to it on the R side:

```
texts_tbl <- sdf_copy_to(sc, texts_df, overwrite = TRUE)
```

4) Tokenize and Vectorize (Bag of Words)

We'll use Spark ML's **Tokenizer + StopWordsRemover + CountVectorizer** to get bag-of-words vectors for the scraped Wikipedia text. These stages turn raw text into numeric features:

- Tokenization splits sentences into words
- The stop-word remover drops high-frequency filler terms, such as "the" or "and"
- The count vectorizer builds a sparse vector describing how often each vocabulary term appears. All three steps can be chained together inside a single Spark ML pipeline.

These three Spark ML stages are exposed in `sparklyr` as:

- `ft_tokenizer()` splits the raw `text` column into individual word tokens so downstream steps operate on consistent pieces of text.
- `ft_stop_words_remover()` drops high-frequency stop words (like "the" or "and") from the token list, cutting noise before featurization.
- `ft_count_vectorizer()` builds a sparse term-frequency vector, counting how often each token appears and mapping documents into numeric feature space.

Question: Apply these three transformations, in the order given as above, to form a `%>%` pipeline with `ml_pipeline()` that takes the `texts_tbl` Spark DataFrame as input and produces a new DataFrame `docs_features` with columns `name` and `features` (the bag-of-words vector).

```
pipeline <- ml_pipeline(sc) %>%
  # Your solution here
```

We can now apply the pipeline to the documents to get their feature vectors:

```
model <- ml_fit(pipeline, texts_tbl)
docs_augmented <- ml_transform(model, texts_tbl)
docs_features <- docs_augmented %>% select(name, features)
docs_features
```

The output shows each document transformed into a numerical representation:

- Each **`features`** vector is a sparse array of length 5,386 (the total vocabulary size), where each position corresponds to one word from the vocabulary and the value at that position indicates how many times that word appears in the document.
- For example, `<dbl [5,386]>` means a numeric vector with 5,386 elements, where most entries are zero (words not in this particular document) and non-zero entries count occurrences of specific vocabulary terms.

This bag-of-words representation allows us to compare documents mathematically: documents with similar word usage patterns will have similar feature vectors.

To see what's inside these feature vectors, we can look up the counts for specific words. The count vectorizer creates a vocabulary mapping, which we can use to find word positions:

```r
# Get the vocabulary from the fitted count vectorizer model
vocab <- model$stages[[3]]$vocabulary

# Helper function to extract counts for specific words from feature vectors
extract_word_counts <- function(docs_df, vocab, words) {
  word_indices <- setNames(match(words, vocab), words)

  result <- docs_df %>%
    select(name, features)

  for (word in words) {
    idx <- word_indices[[word]]
    col_name <- paste0(word, "_count")
    result <- result %>%
      mutate(!!col_name := map_dbl(features, ~ if (!is.na(idx)) .x[idx] else 0))
  }

  result %>% select(-features)
}

# Extract counts for some interesting words
interesting_words <- c("poetry", "actor", "theatre", "star")

docs_augmented %>%
  collect() %>%
  extract_word_counts(vocab, interesting_words)
```

This shows how many times words like "poetry", "actor", "theatre", and "star" appear in each document, revealing the content differences between the Shatner and Shakespeare pages.

5) Represent the query "wrote poetry" with the same featurizer

Our search engine will score how well a short query matches each document. To keep the comparison fair, we must run the query text through the same pipeline so it lives in the exact same feature space as the documents.

Question: Complete the code below to copy the query table to Spark and transform it using the fitted model. You'll need to apply `ml_transform()` with the trained `model` to get the query's feature vector.

```r
query_df <- tibble(name = "query", text = "wrote poetry")
query_tbl <- # Your solution here
query_augmented <- # Your solution here
query_features <- # Your solution here
query_features
```

Question: Refactor the code above into a reusable function called `featurize_query()` that takes two parameters: `query_name` (a string identifier) and `query_text` (the query string), and returns the Spark DataFrame produced by `ml_transform()`.

```r
featurize_query <- function(query_name, query_text) {
  # Your solution here
}
```

```r
# Test the function
featurize_query("query", "wrote poetry") %>% select(name, features)
```

5b) Represent the query "Star Trek" with the same featurizer

We can apply the function for any other query - in this case “Star Trek”:

```r
query_df2 <- tibble(name = "query_star_trek", text = "Star Trek")
query_star_trek_augmented <- featurize_query("query_star_trek", "Star Trek")
query_star_trek_augmented %>% select(name, features)
```

6) Compare documents to the query with a simple token-overlap score

Finally, we need a similarity metric. We’ll use the **Jaccard similarity** (also called the Jaccard index), which measures how similar two sets are by comparing their overlap. It’s calculated as:

$$\text{Jaccard}(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

where $A \cap B$ is the intersection (words that appear in both sets) and $A \cup B$ is the union (all unique words in either set). The result ranges from 0 (no words in common) to 1 (identical sets of words). For example:

- Document tokens: {“wrote”, “poetry”, “plays”, “theatre”}
- Query tokens: {“wrote”, “poetry”}
- Intersection: {“wrote”, “poetry”} $\rightarrow$ size $= 2$
- Union: {“wrote”, “poetry”, “plays”, “theatre”} $\rightarrow$ size $= 4$
- Jaccard similarity $= 2/4 = 0.5$

This metric works well for search because it rewards documents that share many query terms while penalizing documents with lots of irrelevant vocabulary.

Now the plan is to explode the token arrays, count their unique values, and join those counts to compute intersections and unions without collecting the raw text back into R. We can invoke Spark SQL functions like `explode()` directly inside dplyr verbs (e.g., `transmute(token = explode(filtered))`) so the code stays tidyverse-flavoured. The object `docs_augmented` below is the Spark DataFrame returned by `ml_transform(model, texts_tbl)` in the previous section.

First, break each document’s filtered token array into one row per token while retaining the document name so you can compare documents to queries token-by-token.

```r
doc_tokens_tbl <- docs_augmented %>%
  transmute(name, token = explode(filtered)) %>%
  distinct()
```

The code `transmute(name, token = explode(filtered))` row-explodes filtered (an array) so you get one row per element of that array, with name repeated and each element placed in token. E.g.: from one row like name = “doc1”, filtered = [“cat”,“dog”] you get two rows: (“doc1”,“cat”), (“doc1”,“dog”).

The final `distinct()` removes duplicate name/token pairs, leaving a tidy table where each row represents a unique token observed in a particular document. This structure makes downstream joins with query tokens straightforward because both sides share the same `(name, token)` shape.

```
# Preview the first few tokens per document once you've created doc_tokens_tbl.
# Example:
# doc_tokens_tbl %>%
#   slice_sample(n = 6) %>%
#   arrange(name, token)
#
# doc_tokens_tbl %>%
#   summarise(total_rows = n()) %>%
#   collect()
```

Question: Compute a Spark table named `doc_token_counts` that stores the number of distinct tokens in each document. You'll group by `name`, call `n_distinct(token)` inside `summarise()`, and rely on these counts later when you form each document/query union for the Jaccard score.

```
doc_token_counts <- # Your solution here
```

```
doc_token_counts %>%
  collect()
```

Your output should look something like this:

```
## # A tibble: 2 x 2
##   name        doc_token_count
##   <chr>                 <dbl>
## 1 shatner                3383
## 2 shakespeare            2531
```

With the documents prepared, define a helper that explodes the query's filtered tokens inside Spark so later joins have a tidy table to work with.

```
extract_query_tokens <- . %>%
    transmute(token = explode(filtered)) %>%
    distinct()
```

We'll also capture the query's token count as a scalar. Keeping this logic in a helper avoids repeating the `summarise()`/`collect()`/`pull()` pattern whenever we score a new query.

```
count_query_tokens <- . %>%
    summarise(query_count = n_distinct(token)) %>%
    collect() %>%
    pull(query_count)
```

Finally, create a helper that computes the document/query token intersections entirely in Spark, returning the overlap size for each document.

```
compute_intersection_counts <- function(doc_tokens_tbl, query_tokens_tbl) {
  doc_tokens_tbl %>%
    inner_join(query_tokens_tbl, by = "token") %>%
    group_by(name) %>%
    summarise(intersection_size = n_distinct(token))
}
```

```r
compute_spark_jaccard <- function(doc_tokens_tbl, query_augmented_tbl) {
  query_tokens_tbl <- extract_query_tokens(query_augmented_tbl)

  query_token_count <- count_query_tokens(query_tokens_tbl)

  intersection_counts <- compute_intersection_counts(doc_tokens_tbl, query_tokens_tbl)

  doc_token_counts %>%
    left_join(intersection_counts, by = "name") %>%
    mutate(
      intersection_size = coalesce(intersection_size, 0L),
      union_size = doc_token_count + !!query_token_count - intersection_size,
      jaccard_similarity_to_query = ifelse(union_size == 0, 0, intersection_size / union_size)
    ) %>%
    select(name, jaccard_similarity_to_query) %>%
    arrange(desc(jaccard_similarity_to_query))
}

# Run the helper against the "wrote poetry" query tokens to produce the ranking.
results <- compute_spark_jaccard(doc_tokens_tbl, query_augmented)
results
```

6b) Compare documents to the "Star Trek" query

The same helper gives us a ranking for the “Star Trek” query. Notice how the overlap score shifts because the vocabulary of each page is different.

```r
results_star_trek <- compute_spark_jaccard(doc_tokens_tbl, query_star_trek_augmented)
results_star_trek
```

7) Cleanup

After finishing any Spark work, it’s good practice to close the connection so system resources are released.

```r
spark_disconnect(sc)
```